SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	DIWAKAR S	**Reg.No.:**	192525209
Department:	AIML	**Date:**	26/8/2026

81%

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 µs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I S.Dwarakesh 19 25 25 2021 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

S.Dwarakesh

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	2	2	2	1	0.5
Q2	2	2	1.5	1	0.5
Q3	2.5	2.5	1.5	1.5	1
Q4	2	2.5	2	1.5	1
Q5	2.5	2	2	1	0.5

RUBRICS FOR CALCULATION: A0.5

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 1.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**DIWAKAR S**	**Reg.No.:**	**192525209**
Department:	**AIML**	**Date:**	**26.08.2026**

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- **CPU utilization must be below 15%**
- **Multiple sensors generate interrupts every 1 ms**
- **Use vectored interrupt handling**
- **Select between memory-mapped and I/O-mapped I/O for optimal latency**

Problem Understanding

A real-time embedded system receives data from multiple sensors. Each sensor can generate an interrupt every **1 ms**. Therefore, the system must respond quickly while keeping **CPU utilization below 15%**. Vectored interrupt handling is required for fast identification of the interrupt source.

Constraint Analysis

- CPU utilization must be **less than 15%**.
- Multiple sensors generate interrupts every **1 ms**.
- **Vectored interrupts** must be used.
- I/O technique should provide low latency.
- The system must handle frequent sensor requests without overloading the CPU.

Proposed Solution

Use **memory-mapped I/O with a vectored interrupt controller**.

Architecture:

```
Sensors → Interrupt Controller → CPU
 Sensors → Memory-Mapped Registers → RAM
CPU ↔ RAM / I/O Registers
```

Each sensor is assigned a unique interrupt vector. When a sensor generates an interrupt, the interrupt controller identifies the sensor and directly provides the corresponding vector address to the CPU.

Memory-mapped I/O is selected because device registers are accessed using normal memory instructions. This provides a simple and efficient interface with low access overhead.

Working

1. Sensors continuously monitor their environment.
2. When a sensor requires CPU attention, it generates an interrupt.
3. The interrupt controller receives the request.
4. The controller identifies the source using the interrupt vector.
5. CPU jumps directly to the corresponding Interrupt Service Routine (ISR).
6. The ISR reads the sensor data from its memory-mapped register.
7. The data is stored in memory.
8. CPU returns to the main program.

CPU Utilization

If interrupt processing is kept short, most sensor processing can be performed using efficient buffering and batch processing. Therefore, CPU utilization can be maintained below **15%**.

Justification

Memory-mapped I/O + vectored interrupts is suitable because it provides fast register access and avoids unnecessary interrupt-source checking. Short ISRs and buffering reduce CPU overhead, allowing the system to satisfy the real-time requirement.

2.Design an I/O subsystem under these constraints:

- **High-speed storage requires throughput > 3 GB/s**
- **Must use NVMe over PCIe**
- **CPU should not handle bulk data transfer**
- **Integrate DMA controller with interrupt notification**

Problem Understanding

The system requires storage throughput greater than 3 GB/s. It must use NVMe over PCIe, while the CPU should not perform bulk data transfers. Therefore, DMA must be used to transfer data directly between storage and memory.

Constraint Analysis

- Storage throughput must be > 3 GB/s.
- Must use NVMe over PCIe.

- CPU should not handle bulk data movement.
- DMA controller must transfer data.
- Interrupt notification should inform the CPU when operations are completed.

Proposed Architecture

```
NVMe SSD
↓
PCIe Interface
↓
DMA Controller
↓
Main Memory (RAM)
↓
CPU
```

Working

1. The CPU sends an I/O request to the NVMe controller.
2. The NVMe controller communicates through the PCIe interface.
3. DMA is configured with the source, destination, and transfer size.
4. DMA transfers data directly between the NVMe device and RAM.
5. CPU is free to perform other operations during the transfer.
6. After completing the transfer, DMA/NVMe generates an interrupt.
7. CPU executes the corresponding interrupt service routine.
8. The application processes the transferred data.

Why DMA?

Without DMA, the CPU would need to repeatedly read and write large amounts of data. This would consume significant CPU time.

With DMA:

NVMe → PCIe → DMA → RAM

The CPU mainly manages the transfer rather than moving every byte.

Justification:

NVMe over PCIe provides high-speed storage communication, while DMA eliminates unnecessary CPU involvement. Interrupt notification allows the CPU to respond only when the transfer is complete. This architecture is appropriate for achieving more than 3 GB/s throughput while keeping CPU overhead low.

3.Construct a multi-device communication architecture with constraints:

- **6 peripherals using USB, 2 high-speed devices using Thunderbolt**
- **Avoid interrupt starvation**
- **Use hardware and software interrupts**
- **Ensure fair bus arbitration**

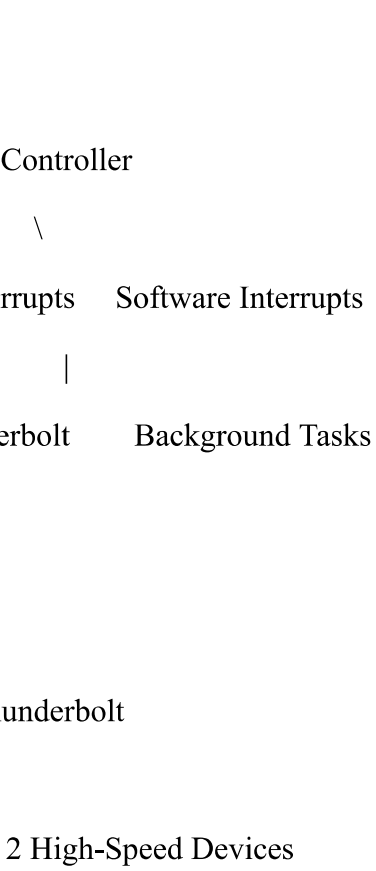
Problem Understanding

The system contains 6 USB peripherals and 2 high-speed Thunderbolt devices. The architecture must prevent interrupt starvation, support hardware and software interrupts, and provide fair bus arbitration.

Constraint Analysis

- 6 USB devices must be connected.
- 2 high-speed Thunderbolt devices must be supported.
- Interrupt starvation must be avoided.
- Hardware and software interrupts must be supported.
- Bus access must be fairly distributed.

Proposed Architecture



Working

1. The six USB devices are connected through a USB hub/controller.
2. The two Thunderbolt devices use their high-speed interface.
3. The bus arbiter controls which device gets access to the communication bus.
4. Hardware interrupts are used for time-critical device events.
5. Software interrupts are used for operating-system or background service requests.
6. A priority-based system can handle urgent events.
7. Round-robin arbitration can be used among devices with similar priority.
8. Interrupt priorities are managed so that low-priority devices are not permanently blocked.

Avoiding Interrupt Starvation

Interrupt starvation can occur when high-priority devices continuously generate interrupts.

To avoid this:

- Use interrupt priorities carefully.
- Limit the time spent in each ISR.
- Use round-robin scheduling for similar-priority devices.
- Temporarily defer non-critical processing.
- Use buffering for frequent USB events.

Fair Bus Arbitration

A round-robin bus arbitration mechanism can give each requesting device an opportunity to access the bus.

Example:

```
USB1 → USB2 → USB3 → ... → Thunderbolt1 → Thunderbolt2 → repeat
```

High-priority traffic can still receive priority when necessary, while fairness mechanisms prevent permanent blocking.

Justification

The proposed architecture supports all eight peripherals while separating high-speed Thunderbolt communication from USB traffic. Interrupt prioritization and fair arbitration prevent starvation and ensure efficient resource sharing.

4.Design a data acquisition system with constraints:

- **Continuous data stream at 500 MB/s**
- **Minimal CPU intervention required**
- **Must compare DMA transfer vs interrupt-driven I/O**
- **Use memory-mapped I/O for device registers**

Problem Understanding

The system continuously receives data at 500 MB/s. Since the data rate is high, the CPU should perform minimum intervention. The system must compare DMA transfer and interrupt-driven I/O and use memory-mapped I/O for device registers.

Constraint Analysis

- Continuous data rate = 500 MB/s.
- CPU intervention must be minimal.
- DMA and interrupt-driven I/O must be compared.
- Device registers must use memory-mapped I/O.

Proposed Architecture

Data Acquisition Device

| 500 MB/s

↓

Memory-Mapped I/O Registers

|

↓

DMA

|

↓

RAM Buffer

|

↓

CPU

DMA vs Interrupt-Driven I/O

SFeature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Large data transfer	Excellent	Less suitable
Transfer speed	High	Lower for continuous streams
CPU overhead	Low	Higher
Suitable for 500 MB/s	Yes	No, generally inefficient
Data movement	DMA controller	CPU/ISR
Best use	Continuous high-speed data	Small or occasional transfers

Working Using DMA

1. The data acquisition device receives continuous data.
2. CPU configures the DMA controller.
3. Device registers are accessed through memory-mapped I/O.
4. DMA transfers incoming data directly into a RAM buffer.
5. CPU does not handle every data item.
6. When a buffer becomes full, DMA generates an interrupt.
7. CPU processes the completed buffer while DMA can continue filling another buffer.

Double Buffering

A double-buffering technique can be used:

```
Device → Buffer A → CPU processing
```

while simultaneously:

```
Device → Buffer B
```

After Buffer A is processed, the roles are exchanged.

This helps maintain continuous data flow.

Justification

DMA is clearly preferable for 500 MB/s continuous data because interrupt-driven I/O would generate a large number of CPU operations. DMA transfers blocks of data efficiently and allows the CPU to concentrate on processing rather than data movement.

5.Construct an interrupt handling mechanism with constraints:

- **Support nested interrupts**
- **Maximum interrupt latency < 10 µs**
- **Use vectored interrupts for high-priority devices**
- **Use software interrupts for background tasks**

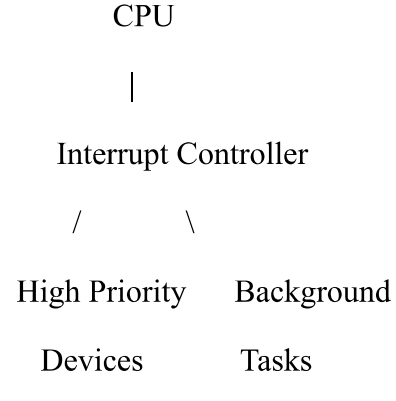
Problem Understanding

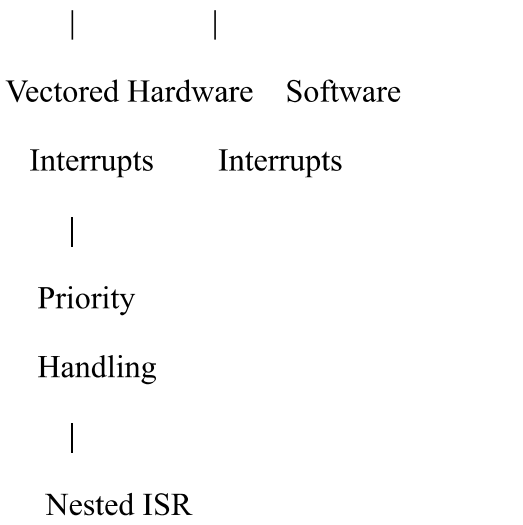
The system must support nested interrupts, maintain maximum interrupt latency below 10 µs, use vectored interrupts for high-priority devices, and use software interrupts for background tasks.

Constraint Analysis

- Nested interrupts must be supported.
- Maximum interrupt latency must be < 10 µs.
- High-priority devices require vectored interrupts.
- Software interrupts should handle background tasks.
- High-priority events must be handled quickly.

Proposed Architecture





Working

1. Devices generate hardware interrupts.
2. The interrupt controller determines the priority.
3. A high-priority interrupt receives immediate attention.
4. The vector identifies the correct ISR.
5. If another higher-priority interrupt occurs while an ISR is executing, nested interrupt handling allows the CPU to temporarily service it.
6. After completing the higher-priority ISR, the CPU returns to the previous ISR.
7. Software interrupts are used for non-critical background operations.

Maintaining <10 µs Latency

The following techniques can be used:

- Use vectored interrupts.
- Keep ISRs short.
- Assign high priority to time-critical devices.
- Avoid long critical sections.
- Save and restore only necessary CPU registers.
- Use efficient interrupt-controller hardware.
- Defer lengthy processing to background tasks.

Justification

Vectored interrupts reduce the time required to identify the interrupt source. Nested interrupts allow urgent events to interrupt lower-priority processing. Software interrupts keep background activities separate from critical hardware events. Together, these mechanisms help achieve the required less-than-10-µs maximum interrupt latency.

Conclusion

The five designs apply the major Computer Architecture I/O concepts required by the assessment: memory-mapped I/O, I/O communication, vectored and hardware/software interrupts, DMA, PCIe, NVMe, USB, and Thunderbolt.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided